Problems on DP: Solutions to some problems in §3 - §5

Graduate Algorithms - Problem Sheet 4

Problems proposed/suggested by Arjun Maneesh Agarwal

Vardhan Kumar Ray

Chennai Mathematical Institute

12 October, 2026

General Bottom-Up DP Template

1. **Define the DP state** $dp[i]$, $dp[i][j]$, ...
2. **Identify base cases** smallest subproblems whose answers are known.
3. **Write the transition** express $dp[\cdots]$ in terms of smaller indices.
4. **Determine computation order** ensure each $dp[\cdots]$ is computed *after* all values it depends on.
5. **Extract the final answer** read off $dp[\cdots]$ at the target state.

Mini-Example: Fibonacci

$$dp[0] = 0, \quad dp[1] = 1 \quad dp[i] = dp[i-1] + dp[i-2], \quad i \geq 2$$

Fill left-to-right; answer is $dp[n]$. Time: $O(n)$, Space: $O(1)$ after rolling-array optimisation. These bounds count arithmetic operations and stored numbers.

Question: Longest Palindromic Subsequence

Problem

Given a string S of length N, find a longest palindromic subsequence of S.

A *palindrome* is a string that equals its own reverse.

E.g. a longest palindromic subsequence of `abcabca` is `abcba`.

Solution: Longest Palindromic Subsequence

1. **State** $dp[i][j]$ = length of a longest palindromic subsequence of $S[i..j]$.
2. **Base cases** $dp[i][i] = 1$; $dp[i][i-1] = 0$ (empty string).
3. **Transition** for $i < j$, $dp[i][j] = \begin{cases} dp[i+1][j-1] + 2 & S[i] = S[j], \\ \max\big(dp[i+1][j],\ dp[i][j-1]\big) & S[i] \neq S[j]. \end{cases}$
4. **Order** increasing length $j - i$.
5. **Answer** $dp[1][N]$; trace the choices back from $(1, N)$ to recover the subsequence.

Mini-Example

$S =$ `abcabca`: $dp[1][7] = 5$, e.g. `abcba`.

Time: $O(N^2)$, Space: $O(N^2)$.

Question: Coin Change

Problem

Given an unlimited supply of coins of positive integer denominations $x_1, x_2, \ldots, x_n$, we wish to make change for a non-negative integer value v; that is, we wish to find a multiset of coins whose total value is v.

Design an $O(nv)$ dynamic programming algorithm for this problem.

Solution: Coin Change

1. **State** $dp[w]$ = minimum number of coins with total exactly w (∞ if impossible).
2. **Base cases** $dp[0] = 0$; initialise all other entries to ∞.
3. **Transition** $dp[w] = 1 + \min_{i:\, x_i \le w} dp[w - x_i]$ (the last coin is x_i); record a minimiser in $choice[w]$ when $dp[w] < \infty$.
4. **Order** $w = 1, 2, \ldots, v$.
5. **Answer** change exists iff $dp[v] < \infty$; read the coins off $choice[\cdot]$, starting from v.

Mini-Example: $x = \{1, 3, 4\}$, $v = 6$

$dp[0..6] = 0, 1, 2, 1, 1, 2, 2$, so $6 = 3 + 3$.

Use $\min \varnothing = \infty$ and $1 + \infty = \infty$.
Time: $O(nv)$, Space: $O(v)$.

Question: Triangle Path (IOI 1994, P1)

Problem

Write a program that calculates the highest sum of numbers passed on a route that starts at the top of a number triangle and ends somewhere on its base.

Each step goes either diagonally down to the left or diagonally down to the right.

For the triangle shown, the answer is 30, via $7 \to 3 \to 8 \to 7 \to 5$.

$$
\begin{array}{ccccccccc}
 & & & & 7 & & & & \\
 & & & 3 & & 8 & & & \\
 & & 8 & & 1 & & 0 & & \\
 & 2 & & 7 & & 4 & & 4 & \\
4 & & 5 & & 2 & & 6 & & 5
\end{array}
$$

Solution: Triangle Path

1. **State** $dp[i][j]$ = largest sum of a route from position (i, j) down to the base.
2. **Base cases** $dp[n][j] = a[n][j]$.
3. **Transition** $dp[i][j] = a[i][j] + \max\big(dp[i+1][j],\, dp[i+1][j+1]\big)$.
4. **Order** rows $n-1, n-2, \ldots, 1$.
5. **Answer** $dp[1][1]$; with the full DP table, follow the child with the larger DP value to recover the route.

Time: $O(n^2)$. Auxiliary space: $O(n)$ for the sum only, or $O(n^2)$ for direct traceback. Update a single rolling row left to right.

Mini-Example

a	dp
7	30
3 8	23 21
8 1 0	20 13 10
2 7 4 4	7 12 10 10
4 5 2 6 5	4 5 2 6 5

Answer 30 via $7 \to 3 \to 8 \to 7 \to 5$.

Question: Wine Barrels

Problem

There are N barrels of wine stored in a narrow passage. Each year, you sell either the first or the last barrel.

The i-th barrel has initial price $P[i]$, and sells for $k \cdot P[i]$ in the k-th year.

What is the maximum possible total profit?

Solution: Wine Barrels

Observation. Assume $N \geq 1$. If barrels $i..j$ remain, the year is $k = N - (j - i)$, so the state needs no year.

1. **State** $dp[i][j]$ = maximum profit from barrels $i..j$, sold from year $N - (j - i)$ on.
2. **Base cases** $dp[i][i] = N \cdot P[i]$.
3. **Transition** for $i < j$, $dp[i][j] = \max\big(k\,P[i] + dp[i+1][j],\ k\,P[j] + dp[i][j-1]\big)$, $k = N - (j - i)$.
4. **Order** increasing length $j - i$.
5. **Answer** $dp[1][N]$.

Time: $O(N^2)$, Space: $O(N^2)$.

Question: Travelling Rockstar

Problem

Given n cities and a table T such that $T[i][j]$ is the time it takes to go from city i to city j.

A rockstar wants to go on a tour covering all n cities. Find a tour that minimises the travel time.

Interpretation used below: visit each city exactly once and return to the starting city.

Note: we want an $O(n^2 2^n)$ algorithm.

Solution: Travelling Rockstar

Let $V = \{1, \ldots, n\}$. A cycle can start at city 1 without loss of generality.

1. **State** $dp[S][j]$ = minimum time of a path from city 1 that visits each city in S exactly once and ends at j.
2. **Base cases** initialise all states to ∞, except $dp[\{1\}][1] = 0$.
3. **Transition** for $1 \in S$ and $j \in S \setminus \{1\}$,

$$dp[S][j] = \min_{i \in S \setminus \{j\}} \big(dp[S \setminus \{j\}][i] + T[i][j]\big).$$

   Record a minimising predecessor for each finite state.
4. **Order** increasing $|S|$.
5. **Answer** for $n \geq 2$, $\min_{j \neq 1}(dp[V][j] + T[j][1])$. For $n = 1$, the answer is 0 (no travel).

Time: $O(n^2 2^n)$, Space: $O(n2^n)$.

Question: Single Source Shortest Path

Problem

Given a weighted graph $G = (V, E)$ with positive weights, a start location S and a target location T, design an algorithm to find the shortest path from S to T.

Solution: Single Source Shortest Path

Let $\delta(v)$ be the distance from S. Then $\delta(S) = 0$ and, for $v \neq S$,

$$\delta(v) = \min_{(u,v)\in E} \big(\delta(u) + w(u,v)\big).$$

Cycles prevent a simple vertex ordering. Add an edge-budget index.

1. **State** $dp[i][v]$ = minimum length of an $S \to v$ walk with at most i edges.
2. **Base cases** $dp[0][S] = 0$ and $dp[0][v] = \infty$ for $v \neq S$.
3. **Transition**
$$dp[i][v] = \min\Big(dp[i-1][v],\ \min_{(u,v)\in E}\big(dp[i-1][u] + w(u,v)\big)\Big).$$
4. **Order** $i = 1, \ldots, |V| - 1$.
5. **Answer** $dp[|V|-1][T]$; ∞ means T is unreachable.

Use $\min \varnothing = \infty$ and $\infty + w = \infty$.

Shortest Path: Correctness and Reconstruction

- A walk with at most i edges either uses at most $i-1$ edges, or ends with an edge (u, v) after a walk with at most $i-1$ edges. This proves the transition.
- With positive weights, removing any cycle reduces the cost. A shortest path therefore uses at most $|V|-1$ edges.
- For traceback, record whether a state copies the preceding layer or uses an incoming edge. Start at $(|V|-1, T)$ and decrease the layer at each step, recording edges when used. Reverse the recorded edges.

Negative weights

The answer remains valid if no negative-weight cycle is reachable from S. For this target alone, it suffices that no negative cycle is reachable from S and can reach T. Otherwise, the shortest-walk value to T is $-\infty$.

Question: Party

Problem

You are organising a party for your office. The office is very hierarchical: people do not feel comfortable attending if their immediate senior is attending, although they have no issues with anyone else. You and your boss's boss can both be invited, but not you and your boss.

You are given a list S of length n, where $S[i]$ lists the immediate underlings of i ($S[j] = -1$ means j is at the bottom of the hierarchy). Assume an acyclic hierarchy with one root who has no boss; every other employee has exactly one immediate boss.

What is the largest number of people you can invite?

Hint: we want an $O(n)$ algorithm.

Solution: Party

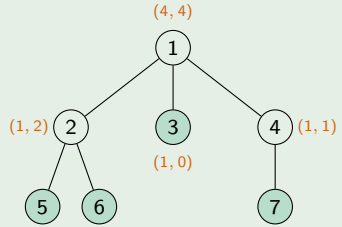
The hierarchy is a rooted tree (treat $S[j] = -1$ as no children). We need a maximum independent set.

1. **State** $in[v]$ / $out[v]$ = most guests from v's subtree, with v invited / not invited.
2. **Base cases** a leaf has $in = 1$, $out = 0$.
3. **Transition** $in[v] = 1 + \sum_{c \in S[v]} out[c]$, $out[v] = \sum_{c \in S[v]} \max(in[c], out[c])$.
4. **Order** children before parents (post-order, or reverse BFS order).
5. **Answer** $\max(in[r], out[r])$ at the root r.

Time: $O(n)$, since $\sum_v |S[v]| = n - 1$.

For a forest with roots R, use the same recurrence and sum $\max(in[r], out[r])$ over $r \in R$.

Mini-Example



Labels are (in, out). Answer 4: the shaded set $\{3, 5, 6, 7\}$.

Question: Polynomial DP (DS-Algo Mock Midsem 2026)

Problem

Given a non-negative integer m, `count_poly(m)` counts the number of polynomials P with coefficients from $\{0, 1, \ldots, 9\}$ such that $P(3) = m$.

Input	Output
$m = 1$	1
$m = 3$	2
$m = 10$	4

Hint: notice that there are no negative coefficients in P.

(a) Write pseudocode implementing `count_poly(m)` in $O(m \log_3 m)$ time.

(b) Argue the correctness of your implementation.

Solution: Polynomial DP - (a) Algorithm

1. **State** $f[v] = \mathit{count_poly}(v)$.
2. **Base cases** $f[0] = 1$ (the zero polynomial).
3. **Transition** for $v \geq 1$:
$$f[v] = \sum_{\substack{0 \leq c \leq \min(9,v) \\ c \equiv v \pmod 3}} f\left[\tfrac{v-c}{3}\right].$$
4. **Order** $v = 1, 2, \ldots, m$.
5. **Answer** $f[m]$.

function $\mathrm{count_poly}(m)$
  $f[0..m] \leftarrow$ array of zeros
  $f[0] \leftarrow 1$
  for $v \leftarrow 1$ **to** m
    for $c \leftarrow 0$ **to** 9
      if $c \leq v$ **and** $(v - c) \bmod 3 = 0$
        $f[v] \leftarrow f[v] + f[(v - c)/3]$
  return $f[m]$

For $m \geq 1$: $O(m)$ arithmetic operations and $O(m)$ stored integers.
This meets the requested $O(m \log_3 m)$ asymptotic bound. For $m = 0$, return 1.

Solution: Polynomial DP - (b) Correctness

Claim. $f[v]$ equals the number of valid P with $P(3) = v$. Induction on v.

- Write $P(x) = c + x\,Q(x)$ with $c = P(0) \in \{0..9\}$. Then Q also has coefficients in $\{0..9\}$, $P(3) = c + 3\,Q(3)$, and $P \mapsto (c, Q)$ is a bijection.
- Coefficients are non-negative, so $Q(3) \geq 0$. Hence $P(3) = v$ iff $c \equiv v \pmod 3$, $c \leq v$ and $Q(3) = (v - c)/3$.
- **Step** ($v \geq 1$). Summing over c gives the transition. Each index $(v - c)/3 \leq v/3 < v$ is already correct.
- **Base** ($v = 0$). Non-negative coefficients and $P(3) = 0$ force $P = 0$, so $f[0] = 1$.

Polynomial DP: Bit Complexity

- If $v \geq 1$ and $P(3) = v$, non-negative coefficients imply $\deg P \leq \lfloor \log_3 v \rfloor$.
- Padding with zero coefficients gives the bound

$$f[v] \leq 10^{1+\lfloor \log_3 v \rfloor}.$$

  Thus each count uses $O(\log(v+1))$ bits.
- The recurrence adds at most four counts per state. Using ordinary integer addition, the algorithm takes $O(m \log(m+1))$ bit operations and stores $O(m \log(m+1))$ bits for $m \geq 1$.

Check for $m = 10$

The four polynomials are $x^2 + 1$, $3x + 1$, $2x + 4$, and $x + 7$.